WARLOCK PROJECT / COLLABORATION SUMMARY

What we built together

A record of the coding editor, live dual-player challenge mode, code checking, and deployment setup developed through our work.

5	2	80 / 20
languages	players per room	correctness / efficiency

The result

Warlock grew from a browser-based code editor into a multiplayer coding challenge experience: players can join a room, work in parallel, run their solutions against challenge cases, and compare results. We also laid out the local execution and cloud deployment pieces needed to operate it.

A clear split of responsibilities

Vercel serves the frontend. **Render** runs the room and WebSocket server. **Piston in Docker** executes code for local development. A deployed website still needs a publicly reachable, safely proxied Piston service for remote submissions.

This summary separates implemented project work from the deployment steps that still depend on an accessible code runner.

FEATURES / IMPLEMENTED WORK

From editor to head-to-head coding

The main product work centered on making coding challenges usable in multiple languages and playable by two people at once.

01 Multi-language editor

Added a language selector for JavaScript, Python, Java, C++, and C#. Editor templates and challenge harnesses are generated for each language.

02 Live dual-player rooms

Built room creation and joining with short room codes, player names, shared challenge state, and live code updates between two players.

03 Submit and compare

Added individual submissions, test-case results, score comparison, and winner selection. The submit control and dual-editor layout were adjusted so the action remains visible.

04 Correctness plus efficiency

The score weights test correctness at 80% and a language-specific efficiency heuristic at 20%. The heuristic provides guidance; it is not a formal complexity proof.

05 Real compilation and runtime checks

Connected the scoring flow to Piston so compiled languages can report compiler errors and all supported languages can execute the challenge harness when the required runtime is installed.

06 Docker-based local runner

Added a Docker Compose service, persistent runtime storage, an installer for the five language runtimes, and a Vite proxy for local browser requests.

Product decisions and fixes

- Kept the frontend and the long-running multiplayer WebSocket process as separate deployable services.
- Added a health endpoint and Render port handling so the dual server can run as a web service.
- Added a production-side message for an unconfigured runner instead of silently sending submissions to a missing local proxy.

DEPLOYMENT / WHAT'S NEXT

How the pieces fit

We worked through Vercel build settings, Render service settings, public WebSocket URLs, and the difference between local Docker execution and cloud execution.

PLAYERS' BROWSER	LINK	SERVICE
Vercel frontend Vite build output: dist	<->	**Piston API** Docker, local during development
Room creation, editors, challenge and scoring UI	runner calls	**Requires an accessible endpoint and a browser-safe proxy in production**
Render Web Service Node WebSocket server	<->	**Room state and live code synchronization**

Deployment knowledge gained

Vercel	Build the Vite frontend with `npm run build`; the output folder is `dist`. The live frontend needs `VITE_DUAL_SERVER_URL` set to the Render WebSocket URL ending in `/dual-ws`, then a redeploy.
Render	Deploy the Node server as a Web Service from the repository root. The service starts with `npm run dual-server` and listens on Render's assigned `PORT`. The `/health` route provides a health check; the root URL can return 404 by design.
Docker / Piston	Run `npm run piston:setup` locally with Docker Desktop running, then `npm run dev`. This local runner is not automatically reachable from Vercel or another player's device.

Current status and remaining dependency

Completed in the project: editor language selection, dual rooms and code synchronization, scoring flow, Render/Vercel configuration, and the local Docker/Piston setup. **Still needed for online code submissions:** a Piston runner hosted somewhere both players' browsers can reach, plus a safe proxy that handles browser access. The Docker service added to this repository binds to the local machine only. The user reported deploying the frontend to Vercel and the dual server to Render; their end-to-end online runner configuration was still being worked through.

A practical milestone: Warlock now has a defined path from writing code, to checking it, to competing live with another player - with the remaining online runner requirement clearly identified.